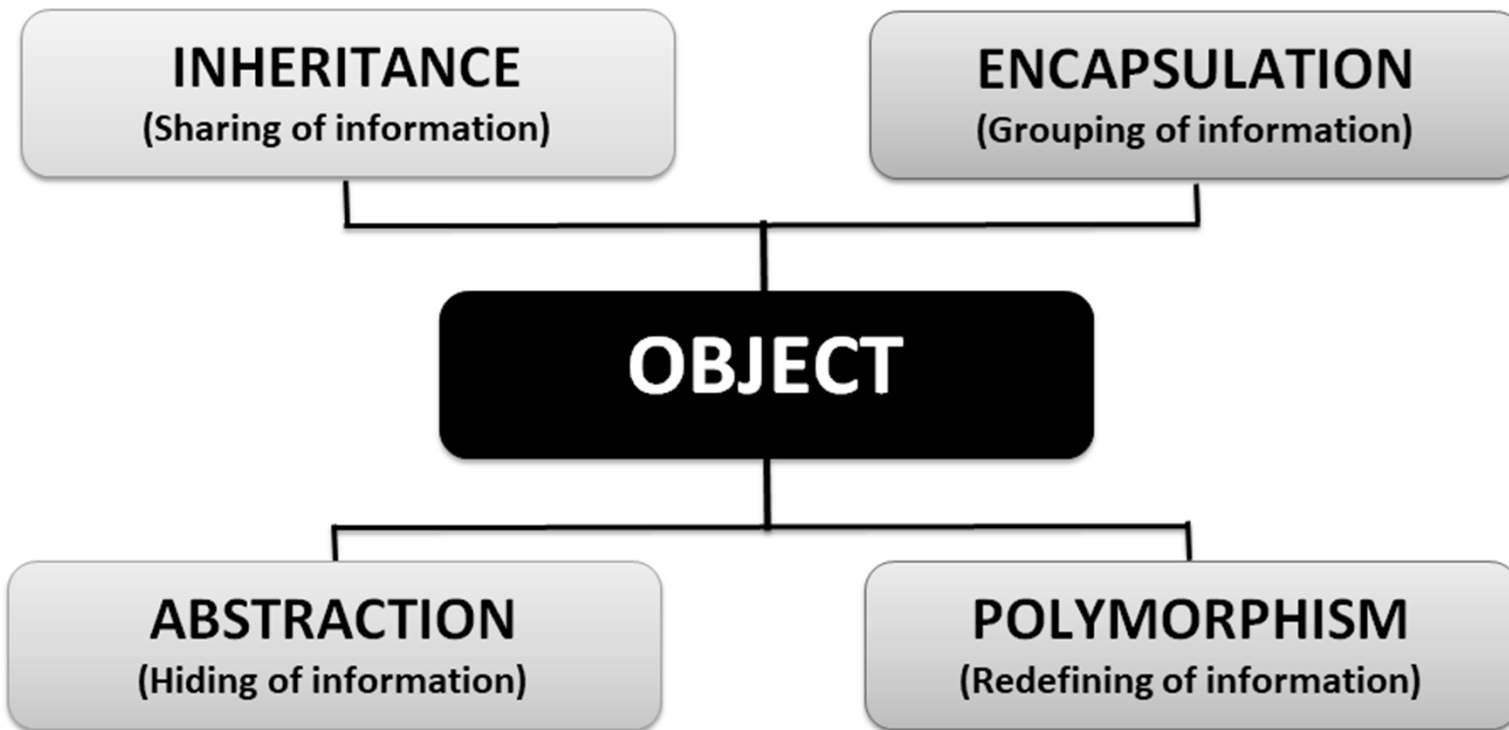


Object Oriented Programming

Four key principles of the OOPS:

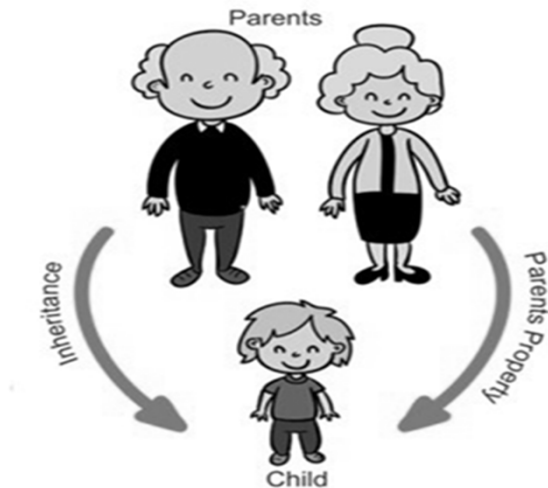


Object Oriented Programming

Inheritance:

Ever heard relatives say, “*You look exactly like your father/mother*”?

That’s **inheritance** in real life — passing characteristics from parents to children.



- Inheritance is a powerful feature in object-oriented programming.
- In programming, inheritance means transferring attributes and behaviors (properties and methods) from a parent (or base) class to a child (or derived) class without rewriting them.
- The child class automatically inherits everything from the parent class, unless it overrides or modifies it.

Object Oriented Programming

Syntax:

class BaseClass:
 Body of base class

class DerivedClass(BaseClass):
 Body of derived class

```
class Employee:  
    pass  
  
class SoftwareEngineer:  
    pass  
  
class Designer:  
    pass
```

A derived class inherits features from the base class, adding new features. This results in the reusability of code.

```
class Employee:  
    pass  
  
class SoftwareEngineer(Employee):  
    pass  
  
class Designer(Employee):  
    pass
```

Object Oriented Programming

```
class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age

class SoftwareEngineer:
    pass

class Designer:
    pass
```

```
E1=Employee("Ram",25)
E1.name
```

'Ram'

```
E2=SoftwareEngineer("Ram",25)
E2.name
```

```
-----
TypeError
Cell In[6], line 1
----> 1 E2=SoftwareEngineer("Ram",25)
      2 E2.name
```

```
class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age
class SoftwareEngineer(Employee):
    pass
class Designer:
    pass

E2=SoftwareEngineer("Ram",25)
E2.name
```

'Ram'

```
class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age
class SoftwareEngineer(Employee):
    pass
class Designer(Employee):
    pass
```

```
E3=Designer("Ram",25)
E3.name
```

'Ram'

Object Oriented Programming

```
class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age

    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    pass

class Designer(Employee):
    pass

E3=Designer("Ram",25)
E3.work()
```

Ram is working

```
E1=Employee("Ram",25)
E1.name
```

'Ram'

```
E1.work()
```

Ram is working

```
E2=SoftwareEngineer("Shyam",28)
E2.name
```

'Shyam'

```
E2.work()
```

Shyam is working

```
E3=Designer("Geeta",20)
E3.name
```

'Geeta'

```
E3.work()
```

Geeta is working

Object Oriented Programming

Properties of Inheritance:

- Inheritance makes objects (classes) special
 - Derive new classes from existing ones (**Inherit**)
 - Extend the definition of existing classes (**Extend**)
 - Override method definitions of existing classes (**Override**)

Object Oriented Programming

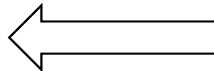
Properties of Inheritance:

- **Extend**
 - Our child class can also have their method.

```
class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    def debug(self):
        print(f"{self.name} is Debugging")

class Designer(Employee):
    def draw(self):
        print(f"{self.name} is Drawing")
```



```
E1=SoftwareEngineer("Ram",28)
E1.work()
```

Ram is working

```
E1. debug()
```

Ram is Debugging

```
E2=Designer("Shyam",25)
E2.work()
```

Shyam is working

```
E2.draw()
```

Shyam is Drawing

Object Oriented Programming

Properties of Inheritance:

- **Extend**
 - Our child class can also have same method.

```
class Employee:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    def work(self):
        print(f"{self.name} is Debugging")

class Designer(Employee):
    pass

E1 = SoftwareEngineer("Ankit", 25)
E1.work()
```

Ankit is Debugging

```
class Employee:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    def work(self):
        super().work()
        print(f"{self.name} is Debugging")

class Designer(Employee):
    pass

E1 = SoftwareEngineer("Ankit", 25)
E1.work()
```

Ankit is working
Ankit is Debugging

Object Oriented Programming

Properties of Inheritance:

- **Extend** (add attributes)

```
class Employee:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    def work(self, extra_charges):
        super().work()
        self.extra_charges = extra_charges
        print(f"{self.name} is Debugging and get Rs. {self.extra_charges}")

class Designer(Employee):
    pass

E1 = SoftwareEngineer("Ankit", 25)
E1.work(25000)
```

Ankit is working
Ankit is Debugging and get Rs. 25000

Object Oriented Programming

Properties of Inheritance:

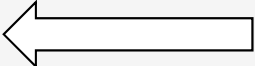
- Extend (Constructor)

```
class Employee:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    def __init__(self, level, salary):
        self.level = level
        self.salary = salary
    def work(self, extra_charges):
        self.extra_charges = extra_charges
        print(f"{self.level} is debugging and takes additional Rs. {self.extra_charges}")

class Designer(Employee):
    pass

E1 = SoftwareEngineer("Senior_Engineer", 25)
E1.work(25000)
```



Senior_Engineer is debugging and takes additional Rs. 25000

Object Oriented Programming

Properties of Inheritance:

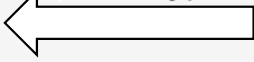
- Extend

```
class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    def __init__(self,name, age, level, salary):
        super().__init__(name,age)
        self.level=level
        self.salary=salary
    def work(self,extra_charges):
        self.extra_charges=extra_charges
        print(f"{self.name} is debugging and takes additional Rs. {self.extra_charges}")

class Designer(Employee):
    pass

E1=SoftwareEngineer("Ram", 25, "Senior_Engineer",25000)
E1.work(25000)
```



Ram is debugging and takes additional Rs. 25000

Object Oriented Programming

Properties of Inheritance:

- **Override:** Method overriding occurs when a child class defines a method with the same name as a method in its parent class.

```
class Employee:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    def __init__(self, name, age, level, salary):
        super().__init__(name, age)
        self.level = level
        self.salary = salary
    def work(self):
        print(f"{self.name} is Coding")
    def debug(self):
        print(f"{self.name} is Debugging")

class Designer(Employee):
    def __init__(self, name, age, level, salary):
        super().__init__(name, age)
        self.level = level
        self.salary = salary
    def work(self):
        print(f"{self.name} is Designing")
    def draw(self):
        print(f"{self.name} is Drawing")
```

```
: E1=Employee("Ram",25)
   E1.work()
```

Ram is working

```
E2=SoftwareEngineer("Shyam",28,"Developer",25000)
```

```
E2.work()
```

Shyam is Coding

```
E3=Designer("Geeta",20,"Designer",20000)
```

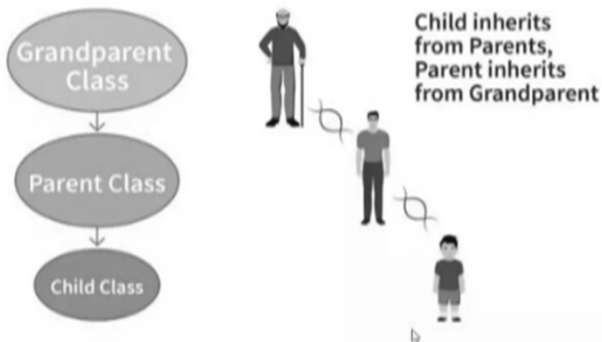
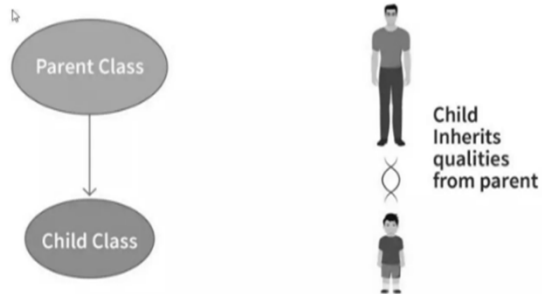
```
E3.work()
```

Geeta is Designing

Object Oriented Programming

Types of Inheritance:

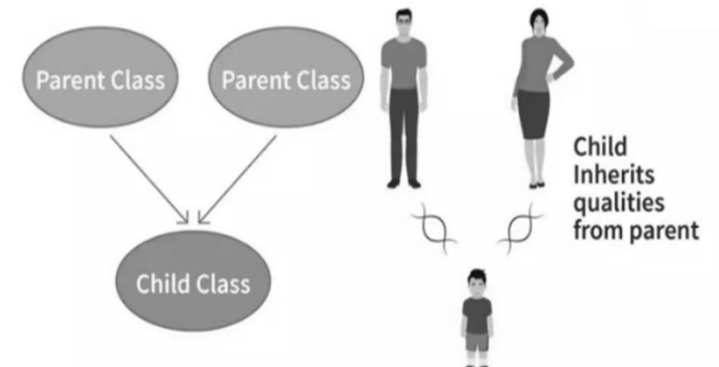
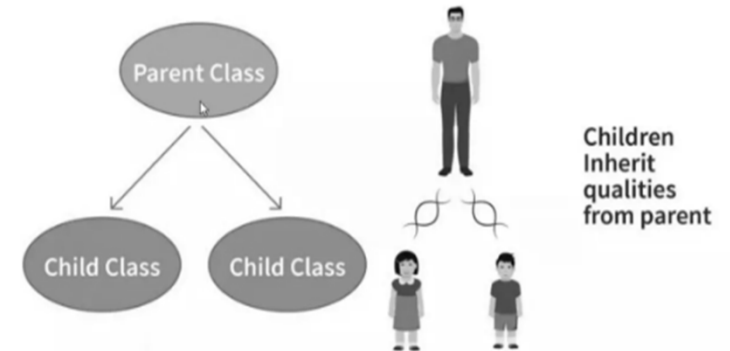
Single Inheritance



Multilevel Inheritance

- **Single:** One child class inherits from one parent class.
- **Multilevel:** A class inherits from another class, which itself inherits from another class (like a chain).
- **Hierarchical:** Multiple child classes inherit from the same parent class.
- **Multiple:** One child class inherits from two or more parent classes.

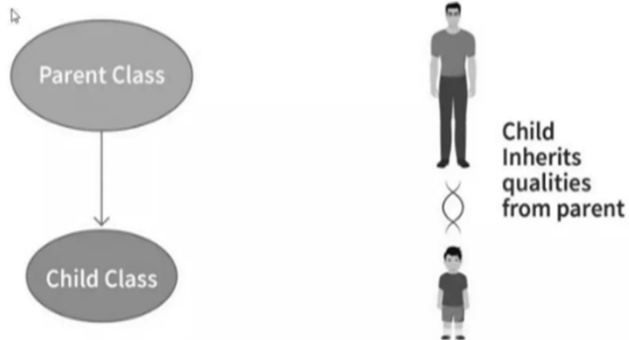
Hierarchical Inheritance



Multiple Inheritance

Object Oriented Programming

Single Inheritance:



```
class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    def debug(self):
        print(f"{self.name} is Debugging")
```

```
E1=SoftwareEngineer("Ram",28)
```

```
E1.work()
```

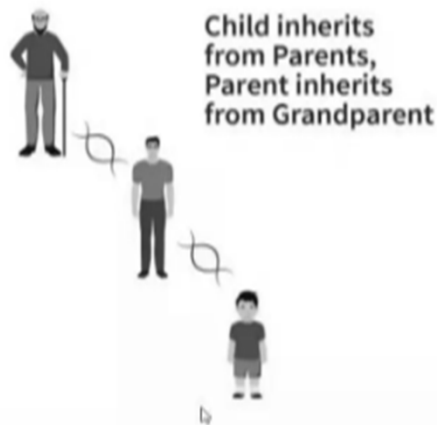
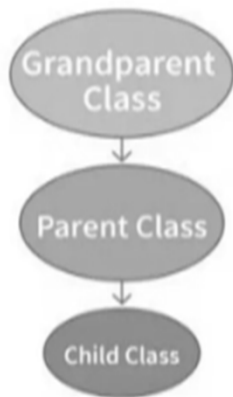
```
Ram is working
```

```
E1.debug()
```

```
Ram is Debugging
```

Object Oriented Programming

Multilevel Inheritance:



```
E1=Designer("Ram",28)
```

```
E1.work()
```

```
Ram is working
```

```
E1.debug()
```

```
Ram is Debugging
```

```
E1.draw()
```

```
Ram is Drawing
```

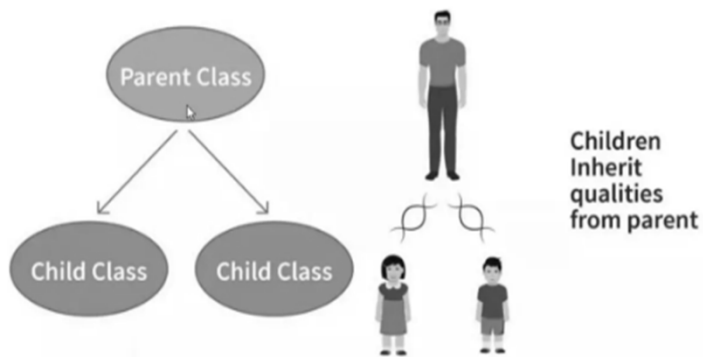
```
class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    def debug(self):
        print(f"{self.name} is Debugging")

class Designer(SoftwareEngineer):
    def draw(self):
        print(f"{self.name} is Drawing")
```

Object Oriented Programming

Hierarchical Inheritance



```
E1=SoftwareEngineer("Ram",28)
E1.work()
```

Ram is working

```
E2=Designer("Shyam",25)
E2.work()
```

Shyam is working

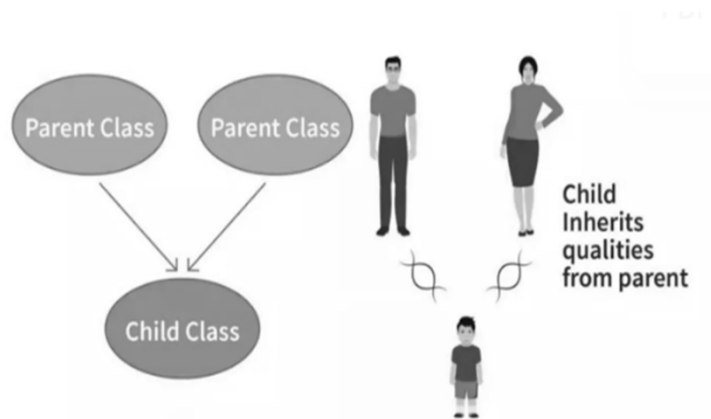
```
: class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def work(self):
        print(f"{self.name} is working")

class SoftwareEngineer(Employee):
    def debug(self):
        print(f"{self.name} is Debugging")

class Designer(Employee):
    def draw(self):
        print(f"{self.name} is Drawing")
```

Object Oriented Programming

Multiple Inheritance:



```
E1=SoftwareEngineer("Ram",25,5000,"Python")
```

```
E1.work()
```

```
Ram is working
```

```
E1.freelance_work()
```

```
Working on freelancer project at rate 5000
```

```
E1.develop()
```

```
Ram develops software using Python
```

```
class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def work(self):
        print(f"{self.name} is working")

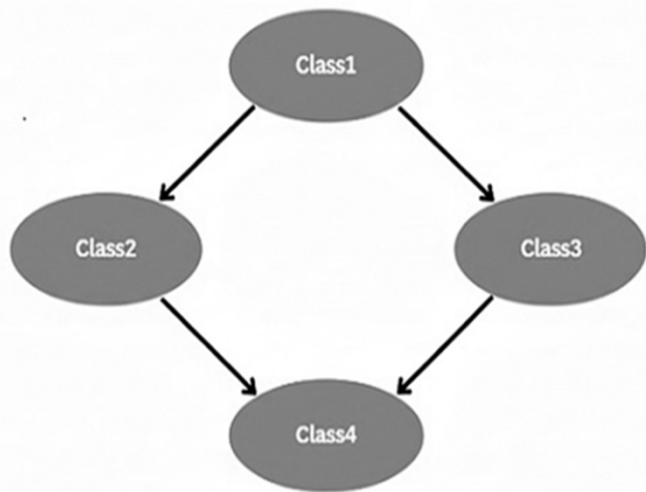
class Freelancer:
    def __init__(self,Project_rate):
        self.project_rate=project_rate
    def freelance_work(self):
        print(f"working on freelancer project at rate {self.project_rate}")

class SoftwareEngineer(Employee,Freelancer):
    def __init__(self, name,age,project_rate,language):
        Employee.__init__(self,name,age)
        Freelancer.__init__(self,project_rate)
        self.language=language

    def develop(self):
        print(f"{self.name} develops software using {self.language}")
```

Object Oriented Programming

Diamond Problem:



- The diamond problem is also known as the "Deadly Diamond of Death".
- It mainly arises when a class inherits from two or more classes that themselves share a common superclass.
- This situation creates ambiguity in the inheritance hierarchy, leading to confusion about which properties or methods of the shared superclass should be inherited.
- The diamond problem occurs in multiple inheritance.

Object Oriented Programming

Diamond Problem:

```
class A:
    def show(self):
        print("Class A")

class B(A):
    def show(self):
        print("Class B")

class C(A):
    def show(self):
        print("Class C")

class D(B):
    pass
```

```
: d=D()
  d.show()
```

Method Resolution Order(MRO):

MRO defines the **order in which Python searches classes** when looking for a method or attribute in inheritance.

```
print(D.mro())
```

```
[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]
```

```
: d=D()
  d.show()
```

Class B

Object Oriented Programming

Diamond Problem with Constructor

Fix using super()

```
class A:
    def __init__(self):
        print("Class A")

class B(A):
    def __init__(self):
        print("Class B")
        A.__init__(self)

class C(A):
    def __init__(self):
        print("Class C")
        A.__init__(self)

class D(B,C):
    def __init__(self):
        print("Class D")
        B.__init__(self)
        C.__init__(self)
```

```
d=D()
```

```
Class D
Class B
Class A
Class C
Class A
```

- A constructor runs twice → duplication.
- This is the Diamond Constructor Problem.

```
class A:
    def __init__(self):
        print("A constructor")

class B(A):
    def __init__(self):
        print("B constructor")
        super().__init__()

class C(A):
    def __init__(self):
        print("C constructor")
        super().__init__()

class D(B, C):
    def __init__(self):
        print("D constructor")
        super().__init__()
```

```
d = D()
```

```
D constructor
B constructor
C constructor
A constructor
```



Object Oriented Programming

Chain break:

```
class A:
    def __init__(self):
        print("A constructor")

class B(A):
    def __init__(self):
        print("B constructor")
        super().__init__()

class C(A):
    def __init__(self):
        print("C constructor")

class D(B, C):
    def __init__(self):
        print("D constructor")
        super().__init__()

d = D()
```

D constructor
B constructor
C constructor

```
: class A:
    def __init__(self):
        print("A constructor")

class B(A):
    def __init__(self):
        print("B constructor")

class C(A):
    def __init__(self):
        print("C constructor")
        super().__init__()

class D(B, C):
    def __init__(self):
        print("D constructor")
        super().__init__()

d = D()
```

D constructor
B constructor

Exercise

Create a Python program to manage a school system using multilevel inheritance:

Requirements:

- Create a base class Person with attributes:
 - name (string)
 - age (integer)
 - Method display() to print name and age.
- Create a subclass Teacher that inherits from Person:
 - Adds an attribute subject
 - Overrides display() to include the subject.
- Create another subclass Student that inherits from Person:
 - Adds attributes student_id and grade
 - Overrides display() to include these details.
- Create a further subclass ExamResult that inherits from Student:
 - Adds a marks attribute (dictionary of subjects and scores)
 - Adds a method display_result() to print all student details including marks.
- Create objects for at least one teacher and one student with exam results, and display their information.

Exercise

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
```

```
class Teacher(Person):
    def __init__(self, name, age, subject):
        super().__init__(name, age)
        self.subject = subject

    def display(self):
        super().display()
        print("Subject:", self.subject)
```

```
class Student(Person):
    def __init__(self, name, age, student_id, grade):
        super().__init__(name, age)
        self.student_id = student_id
        self.grade = grade
    def display(self):
        print("Student Information")
        super().display()
        print("Student ID:", self.student_id)
        print("Grade:", self.grade)
```

```
class ExamResult(Student):
    def __init__(self, name, age, student_id, grade, marks):
        super().__init__(name, age, student_id, grade)
        self.marks = marks

    def display_result(self):
        super().display()
        print("Marks:")
        for subject, score in self.marks.items():
            print(subject, ":", score)
```

```
t1 = Teacher("Mr. Sharma", 40, "Mathematics")
print("Teacher Information")
t1.display()

print("\n")

marks = {"Math": 85, "Science": 90, "English": 88}
s1 = ExamResult("Ankit", 18, "S101", "12th", marks)

print("Student Exam Result")
s1.display_result()
```

```
----- Teacher Details -----
Teacher Information
Name: Mr. Sharma
Age: 40
Subject: Mathematics
```

```
----- Student Exam Result -----
Exam Result
Student Information
Name: Rahul
Age: 16
Student ID: S101
Grade: 10th
Marks:
Math : 92
Science : 88
English : 85
```

Exercise

Create a Python program for a vehicle rental system:

Requirements:

- Create a base class Vehicle with attributes:
 - brand (string)
 - model (string)
 - Method vehicle_info() to display vehicle brand and model.
- Create another base class Rentable with:
 - Attribute price_per_day (integer/float)
 - Method rent_info() to display rental price per day.
- Create a subclass Car that inherits from both Vehicle and Rentable:
 - Adds seats (integer)
 - Method display() should show all car details.
- Create a subclass Bike that inherits from both Vehicle and Rentable:
 - Adds type_bike (string, e.g., "Mountain", "Road")
 - Method display() should show all bike details.
- Create objects for at least one car and one bike, and display their rental information.